

[Home](#) > [Blog](#) > [Elasticsearch](#) >

CRUD Operations & Sorting Documents

Web applications and webpages benefit greatly from flexible search and indexing capabilities. ElasticSearch has grown in popularity because of the ease with which it can be configured and the tremendous flexibility with which it can be used to obtain and interact with data via HTTP protocols. Elasticsearch CRUD operations, as well as flexible search, indexing, and sorting of documents, are covered in this blog.

Rating: 4.8 ★★★★★

👁 5512

GET TRAINED AND CERTIFIED

Flexible searching and indexing for web applications and sites is almost always useful and sometimes absolutely essential. While there are many complex solutions that manage data and allow you to retrieve and interact with it through HTTP methods, ElasticSearch has gained popularity due to its easy configuration and incredible malleability.

Elasticsearch is an open-source search engine built on top of Apache Lucene, a full-text search-engine library.

Table of content - CRUD Operations & Sorting Documents

- [Basic Crud](#)
- [Index API](#)
- [Sample JSON object](#)
- [Getting by ID](#)
- [Deleting Documents](#)



- [JSON objects in documents](#)

Basic Crud

Crud stands for create, read, update, and delete. These are all operations that are needed to effectively administer persistent data storage. Luckily, these also have logical equivalents in HTTP methods, which makes it easy to interact using standard methods. The CRUD methods are implemented by the HTTP methods POST, GET, PUT, and DELETE respectively.

In order to use Elasticsearch for anything useful, such as searching, the first step is to populate an index with some data. This process is known as indexing.

Learn how to use Elasticsearch, from beginner basics to advanced techniques, with online video tutorials taught by industry experts. Enroll in Our [Elasticsearch Certification Course](#) Today!

Index API

In [ElasticSearch](#), indexing corresponds to both “Create” and “Update” in CRUD – if we index a document with a given type and ID that doesn’t already [REST](#) exist it’s inserted. If a document with the same type and ID already exists, it’s overwritten.[JSON](#)

From our perspectives as users of Elasticsearch, a document is a object. As such a document can have fields in the form of JSON properties. Such properties can be values such as strings or numbers, but they can also be other JSON objects.

In order to create a document, we make a PUT request to the API to a URL made up of the index name, type name, and ID. That is [https://localhost:9200/\[index\]/\[type\]/\[id\]](https://localhost:9200/[index]/[type]/[id]) and includes a JSON object as the PUT data.

Index and type are required while the id part is optional. If we don’t specify an ID Elasticsearch will generate one for us. However, if we don’t specify an id we should use POST instead of PUT. The index name is arbitrary. If there isn’t an index with that name on the server already one will be created using the default configuration.

Related Page: [Updating Document Using Elasticsearch Update API](#)

As for the type name it too is arbitrary. It serves several purposes, including:

- Each type has its own ID space.
- Different types can have different mappings (“schema” that defines how properties/fields should be indexed).
- Although it’s possible, and common, to search over multiple types, it’s easy to search only for one or more specific type(s).



Indexes, types and documents

If you’re new to Elasticsearch the terminology and concepts can sometimes be confusing. When I first encountered ES I had primarily worked with relational databases before. While they aren’t the same I sometimes like to think of indexes in ES as databases when working with MySQL or MS SQL Server. When doing so types in Elasticsearch corresponds to tables and documents to individual rows.

Let’s index something! We can put just about anything into our index as long as it can be represented as a single JSON object. For the sake of having something to work with we’ll be indexing, and later searching for, movies. Here’s a classic one:

Sample JSON object

```
{  
  "title": "The Godfather",  
  "director": "Francis Ford Coppola",  
  "year": 1972  
}
```

To index the above JSON object we decide on an index name (“movies”), a type name (“movie”) and an ID (“1”) and make a request following the pattern described above with the JSON object in the body.

A request that indexes the sample JSON object as a document of type ‘movie’ in an index named ‘movies’

```
curl -XPUT "http://localhost:9200/movies/movie/1" -d'  
{  
  "title": "The Godfather",  
  "director": "Francis Ford Coppola",  
  "year": 1972  
}'
```





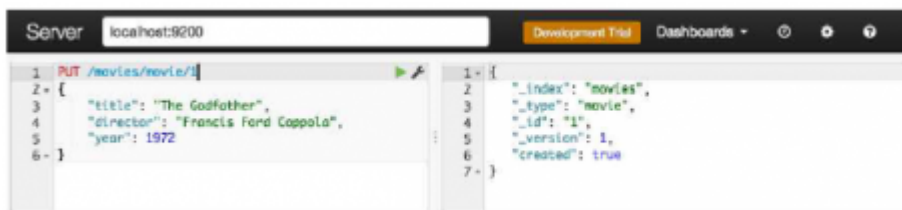
Don't we have to create the index first?

When indexing a document Elasticsearch will automatically create the index (in the example above named "movies") if it doesn't already exist. When doing so it will create the index with some default settings and mappings. We'll see later how to explicitly create indexes and how to specify settings when doing so.

Execute the above request using cURL or paste it into sense and hit the green arrow to run it. After doing so, given that Elasticsearch is running, you should see a response looking like this:

Response from Elasticsearch to the indexing request.

```
{
  "_index": "movies",
  "_type": "movie",
  "_id": "1",
  "_version": 1,
  "created": true
}
```



The request for, and result of, indexing the movie in Sense.

As you see, the response from Elasticsearch is also a JSON object. Its properties describe the result of the operation. The first three properties simply echo the information that we specified in the URL that we made the request to. While this can be convenient in some cases it may seem redundant. However, remember that the ID part of the URL is optional and if we don't specify an ID the `_id` property will be generated for us and its value may then be of great interest to us.

The fourth property, `_version`, tells us that this is the first version of this document (the document with type "movie" with ID "1") in the index. This is also confirmed by the fifth property, `"created"`, whose value is true.



Frequently Asked [Elasticsearch Interview Questions & Answers](#)

Now that we've got a movie in our index let's look at how we can update it, adding a list of genres to it. In order to do that we simply index it again using the same ID. In other words, we make the

exact same indexing request as as before but with an extended JSON object containing genres.

Indexing request with the same URL as before but with an updated JSON payload.

```
curl -XPUT "http://localhost:9200/movies/movie/1" -d'
{
  "title": "The Godfather",
  "director": "Francis Ford Coppola",
  "year": 1972,
  "genres": ["Crime", "Drama"]
}'
```

This time the response from Elasticsearch looks like this:

The response after performing the updated indexing request.

```
{
  "_index": "movies",
  "_type": "movie",
  "_id": "1",
  "_version": 2,
  "created": false
}
```

Not surprisingly the first three properties are the same as before. However, the `_version` property now reflects that the document has been updated as it now has 2 a version number. The `created` property is also different, now having the value `false`. This tells us that the document already existed and therefore wasn't created from scratch.

It may seem that the `created` property is redundant. Wouldn't it be enough to inspect the `_version` property to see if its value is greater than one? In many cases that would work. However, if we were to delete the document the version number wouldn't be reset meaning that if we later indexed a document with the same ID the version number would be greater than one.



So, what's the purpose of the `_version` property then? While it can be used to track how many times a document has been modified it's primary purpose is to allow for optimistic concurrency control.

If we supply a version in indexing requests Elasticsearch will then only overwrite the document

if the supplied version is the same as for the document in the index. To try this out add a version query string parameter to the URL of the request with "1" as value, making it look like this:

Indexing request with a 'version' query string parameter.

```
curl -XPUT "http://localhost:9200/movies/movie/1?version=1" -d'
{
  "title": "The Godfather"
}'
```

Now the response from Elasticsearch is different. This time it contains an error property with a message explaining that the indexing didn't happen due to a version conflict.

Response from Elasticsearch indicating a version conflict.

Related Page: [Curl Syntax In Elasticsearch With Examples](#)

Getting by ID

We've seen how to indexing documents, both new ones, and existing ones, and have looked at how Elasticsearch responds to such requests. However, we haven't actually confirmed that the documents exist, only that ES tells us so.

So, how do we retrieve a document from an Elasticsearch index? Of course, we could search for it. However, that's overkill if we only want to retrieve a single document with a known ID. A simpler and faster approach is to retrieve it by ID.

In order to do that we make a GET request to the same URL as when we indexed it, only this time the ID part of the URL is mandatory. In other words, in order to retrieve a document by ID from Elasticsearch we make a GET request to [HTTP://LOCALHOST:9200/](http://localhost:9200/). Let's try it with our movie using the following request:



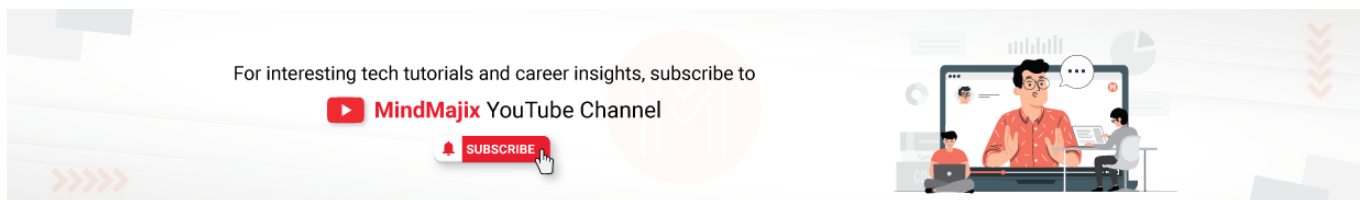
GET request for retrieving the movie with ID 1.

```
curl -XGET "http://localhost:9200/movies/movie/1"
```

You should see a result like this:

```
{
  "_index": "movies",
  "_type": "movie",
  "_id": "1",
  "_version": 2,
  "found": true,
  "_source": {
    "title": "The Godfather",
    "director": "Francis Ford Coppola",
    "year": 1972,
    "genres": [
      "Crime",
      "Drama"
    ]
  }
}
```

As you can see the result object contains similar meta data as we saw when indexing, such as index, type and version. Last but not least it has a property named `_source` which contains the actual document body. There's not much more to say about GET as it's pretty straightforward. Let's move on to the final CRUD operation.



Deleting documents

In order to remove a single document from the index by ID we again use the same URL as for indexing and retrieving it, only this time we change the HTTP verb to DELETE.

Request for deleting the movie with ID 1.

```
curl -XDELETE "https://localhost:9200/movies/movie/1"
```

The response object contains some of the usual suspects in terms of metadata, along with a property named `"_found"` indicating that the document was indeed found and that the operation was successful

Response to the DELETE request.

```
{
  "found": true,
  "_index": "movies",
  "_type": "movie",
  "_id": "1",
  "_version": 3
}
```

If we, after executing the DELETE request, switch back to GET we can verify that the document has indeed been deleted:

Response when making the DELETE request a second time.

```
{
  "_index": "movies",
  "_type": "movie",
  "_id": "1",
  "found": false
}
```

JSON objects in documents

In the examples in this chapter, as well as throughout most of this tutorial, we use fairly simple JSON objects as documents in order to keep the examples short. However, it's worth pointing out that Elasticsearch supports nested JSON objects. For instance, instead of a string, we could have represented the director's property in our movie as a complex object, like this:



```
{
  "title": "The Godfather",
  "director": {
    "givenNames": ["Francis", "Ford"],
    "surNames": ["Coppola"]
  },
  "year": 1972
}
```

Or, like this:

```
{
  "title": "The Godfather",
  "director": {
    "givenNames": ["Francis", "Ford"],
    "surNames": ["Coppola"]
  },
  "year": 1972
}
```

Or, like this:

```
curl -XPUT "https://localhost:9200/movies/movie/1" -d'
{
  "title": "The Godfather",
  "director": {
    "givenName": "Francis Ford",
    "surName": "Coppola",
    "awards": [{
      "name": "Oscar",
      "type": "Director",
      "year": 1974,
      "movie": "The Godfather Part II"
    }]
  },
}
```



```
"year": 1972  
'
```

Explore Elasticsearch Sample Resumes! Download & Edit, Get Noticed by Top Employers! [Download Now!](#)

ElasticSearch Quiz questions

Warm up your Interview preparation with us. Take a quiz and break the buzz.

1. Who is the developer of Elasticsearch?

☐ Elastic NV

☐ Elastic AB

☐ Elastic FB

☐ Elastic ZV

2. What is the default Elasticsearch log level?

☐ Error

☐ Trace

☐ Warn

☐ Info



3. What is the default Elasticsearch communication port?

☐ 9300/tcp

☐ 3000/ip

☐ 9300/utp

☐ 3000/tcp

4. Which of the following is true about Elasticsearch?

☐ Elasticsearch is a real-time distributed and open source full-text search and analytics engine.

☐ Elasticsearch is scalable up to petabytes of structured and unstructured data

☐ Both A and B

☐ None of the above

5. Which of the below is a set of fields defined in JSON format in a certain way?

☐ Node

☐ Shard

☐ Replicas

